Tech Challenge - Fase 2: Otimização de modelos de diagnóstico (AVC)

- Curso: Pós Tech IA para Devs
- Turma: 8IADT
- Funcional: RM369853

Visão Geral do Projeto

Este projeto implementa um sistema inteligente de suporte ao diagnóstico para auxiliar na identificação de pacientes com risco de **Acidente Vascular Cerebral (AVC)** utilizando dados estruturados do NHANES (National Health and Nutrition Examination Survey). O sistema combina **Machine Learning clássico** com **otimização por Algoritmo Genético (GA)** para busca de hiperparâmetros e **integração com LLMs** para interpretação dos resultados em linguagem natural, apoiando (mas não substituindo) decisões clínicas.

Objetivo

Construir uma solução com foco em IA para processamento de dados médicos, aplicando fundamentos essenciais de Machine Learning (ML), algoritmos genéticos e LLMs, demonstrando:

- Exploração e tratamento de dados médicos reais
- Pipeline robusto de pré-processamento
- Modelagem com múltiplas técnicas de classificação
- **Otimização de hiperparâmetros via Algoritmo Genético (GA)** com scorer customizado (F-beta, β=1.5) para maximizar Recall sem degradar Precision
- **Integração com LLMs** para geração de explicações em linguagem natural dos diagnósticos
- Interpretação e comunicação de resultados

Links

Fase 1

- Fase 1 — Vídeo de apresentação do projeto;
- Fase 1 — Notebook Tech Challenge;
- EXTRA - Fase 1 — Notebook Computer Vision CNN;

Fase 2

- Fase 2 — Vídeo de apresentação do projeto
- Fase 2 — Notebook Tech Challenge;

Geral

- Repositório do GitHub;

Instruções de Execução

Pré-requisitos

- Python 3.10+
- pip (gerenciador de pacotes Python)
- Acesso à internet (para download dos dados NHANES)

Instalação e Configuração

1. **Clonar ou acessar o repositório:**

```
cd /home/user/workspaces/fiap-pos-tech-ia-para-devs/01-aulas-gravadas/01-welcome-to-ia-para-devs/07-tech-challenge
```

3. **Criar e ativar um ambiente virtual (recomendado):**

```
python -m venv venv
source venv/bin/activate  # No Windows: venv\Scripts\activate
```

6. **Instalar dependências:**

```
pip install -r requirements.txt
```

8. **Executar o notebook:**

```
jupyter notebook tech-challenge-fase-1.ipynb
```

Ou, se usar JupyterLab:

```
jupyter lab tech-challenge-fase-1.ipynb
```

10. **Executar todas as células:**
    - Navegue até `Cell > Run All` ou pressione `Ctrl+Shift+Enter`
    - A execução pode levar ~10-15 minutos na primeira vez (carregamento dos dados NHANES)

API REST para Predições

Executando a API FastAPI

Após treinar o modelo no notebook e gerar o arquivo `pipe_lr_model.pkl`, você pode iniciar a API para realizar predições em tempo real:

```
# Certifique-se de estar no diretório do projeto com o ambiente virtual
ativado
python -m fastapi dev main.py
```

A API estará disponível em: **http://localhost:8000**

Documentação Interativa (Swagger UI)

Acesse a interface interativa para testar os endpoints:

- **Swagger UI:** http://localhost:8000/docs
- **ReDoc:** http://localhost:8000/redoc

Endpoints Disponíveis

1. GET / — Health Check

Verifica se a API está funcionando.

Request:

```
curl http://localhost:8000/
```

Response:

```
{
  "Hello": "World"
}
```

2. POST /predict — Predição de Risco de AVC

Recebe dados de um paciente e retorna a predição de risco de AVC.

Request Body (JSON):

```
{
  "age": 67,
  "sbp": 128.0,
  "hba1c": 9.2,
  "bmi": 32.0,
  "gender": 1,
  "married": 1.0,
  "high_bp": 1,
  "chf": 0,
  "occupation": 5.0,
  "smoking": 1
}
```

Descrição dos Campos:

Campo	Tipo	Descrição	Valores
`age`	int	Idade em anos	0-150
`sbp`	float	Pressão arterial sistólica (mmHg)	72-228
`hba1c`	float	Hemoglobina glicada (%)	-
`bmi`	float	Índice de Massa Corporal	-
`gender`	float	Gênero	1 = Masculino, 0 = Feminino
`married`	float	Estado civil	1 = Já foi casado, 0 = Nunca casou
`high_bp`	float	Histórico de hipertensão	1 = Sim, 0 = Não
`chf`	float	Insuficiência cardíaca congestiva	1 = Sim, 0 = Não
`occupation`	float	Situação profissional	1-5 (categorias)
`smoking`	float	Histórico de tabagismo	1 = Sim, 0 = Não

Response (JSON):


```json
{
  "prediction_stroke": 0,
  "probability_no_stroke": 0.8234,
  "probability_stroke": 0.1766,
  "input": {
    "age": 67,
    "sbp": 128.0,
    "hba1c": 9.2,
    "bmi": 32.0,
    "gender": 1,
    "married": 1.0,
    "high_bp": 1,
    "chf": 0,
    "occupation": 5.0,
    "smoking": 1
  }
}
```

Interpretação da Response:

- `prediction_stroke`: **0** = Sem risco de AVC, **1** = Com risco de AVC
- `probability_no_stroke`: Probabilidade de NÃO ter AVC (0-1)
- `probability_stroke`: Probabilidade de TER AVC (0-1)
- `input`: Eco dos dados enviados para validação

Integração com Sistemas Clínicos

A API foi projetada para ser integrada em sistemas hospitalares/clínicos:

1. **Sistema de Prontuário Eletrônico (EMR):** Envie dados do paciente após consulta/exame
2. **Sistema de Triagem:** Use para priorizar pacientes de alto risco
3. **Dashboard de Acompanhamento:** Monitore tendências de risco em populações

⚠️ **IMPORTANTE - USO CLÍNICO:**

- Esta API é uma **ferramenta de apoio à decisão**, NÃO um substituto do julgamento médico
- Sempre confirme casos de alto risco com avaliação clínica especializada e exames de imagem (CT/MRI)
- Considere falsos positivos/negativos nas decisões clínicas

Estrutura de Arquivos

```
tech-challenge-fase-1/
├── tech-challenge-fase-1.ipynb    # Notebook principal com EDA,
modelagem e avaliação
├── main.py                         # API FastAPI para predições em tempo
real
├── pipe_lr_model.pkl              # Modelo Logistic Regression treinado
(serializado)
├── requirements.txt                # Dependências do projeto
└── README.md                       # Este arquivo
```

Resultados Obtidos

Tech Challenge 2

Bibliotecas Utilizadas (Algoritmo Genético)

Biblioteca	Versão	Função
`sklearn-genetic-opt`	0.12.0	Otimização de hiperparâmetros via Algoritmo Genético (`GASearchCV`)
`scikit-learn`	1.7.2	Pipelines, modelos (LR, RF), métricas, pré-processamento e validação cruzada
`pandas`	2.3.3	Manipulação e análise de dados tabulares
`numpy`	2.3.5	Operações numéricas e vetorizadas
`matplotlib`	3.10.7	Visualização de gráficos (barras, evolução de fitness, permutação)
`seaborn`	0.13.2	Visualizações estatísticas complementares
`missingno`	0.5.2	Diagnóstico visual de dados ausentes
`fastapi`	0.128.0	API REST para predições em tempo real (produtização)

Otimização de Hiperparâmetros com Algoritmo Genético

Na segunda fase, foram implementados **3 experimentos de otimização de hiperparâmetros por Algoritmo Genético (GA)**, combinando abordagens com biblioteca (`sklearn-genetic-opt`) e implementações manuais (*from scratch*), aplicados a dois modelos de classificação: **Logistic Regression** e **Random Forest**.

O GA evolui populações de configurações de hiperparâmetros ao longo de gerações, aplicando seleção por torneio, crossover uniforme, mutação e elitismo para encontrar combinações que maximizem o desempenho do modelo.

Scorer Customizado

F-beta (β=1.5) com penalidade para modelos degenerados:

```
def custom_scorer(y_true, y_pred):
    prec = precision_score(y_true, y_pred, zero_division=0)
    if prec < 0.3:
        return 0.0  # penaliza soluções triviais
    return fbeta_score(y_true, y_pred, beta=1.5, zero_division=0)
```

- **β=1.5** privilegia Recall (detectar casos de AVC) sobre Precision, sem ignorar falsos positivos
- **Penalidade (prec < 0.3):** impede soluções degeneradas que classificam tudo como positivo (recall=1.0, precision~0.5, F1~0.66)

Espaço de Busca dos Hiperparâmetros

Logistic Regression:

Hiperparâmetro	Tipo	Intervalo
`C`	Contínuo (log-uniform)	0.001 – 100
`penalty`	Categórico	l1, l2
`solver`	Categórico	liblinear, saga
`max_iter`	Inteiro	100 – 2000

Random Forest:

Hiperparâmetro	Tipo	Intervalo
`n_estimators`	Inteiro	50 – 300
`max_depth`	Inteiro	3 – 20
`min_samples_split`	Inteiro	2 – 15
`min_samples_leaf`	Inteiro	1 – 8
`max_features`	Categórico	sqrt, log2, None

Configuração dos 3 Experimentos GA

Parâmetro	Exp 1 (`sklearn-genetic-opt`)	Exp 2 (GA Manual)	Exp 3 (GA Manual)
Implementação	Biblioteca `GASearchCV`	From scratch	From scratch
População	15	30	10
Gerações	10	15	20
Taxa de crossover	0.8	0.9	0.7
Taxa de mutação	0.1	0.05	0.25
Seleção	Torneio (k=3)	Torneio (k=5)	Torneio (k=3)
Elitismo	Sim	Top 2	Top 1
Estratégia	Balanceado	Exploração conservadora	Exploração agressiva

Melhores Hiperparâmetros Encontrados

Logistic Regression (Exp 1 — vencedor):

- `C` = 0.00198, `penalty` = l2, `solver` = liblinear, `max_iter` = 613
- Melhor fitness (treino CV): **0.8045**

Random Forest (Exp 3 — vencedor):

- `n_estimators` = variável, `max_depth` = 10, `min_samples_split` = 9, `min_samples_leaf` = 7, `max_features` = log2
- Melhor fitness (treino CV): **0.7906**

Convergência dos Algoritmos Genéticos

- **LR Exp 1:** convergiu na geração 9/10 (fitness estabilizou em 0.8045)
- **RF Exp 1:** convergiu na geração 9/10 (fitness estabilizou em 0.7900)
- **LR Exp 2 (pop=30):** atingiu 0.8045 na geração 12/15 — convergência mais lenta pela população maior
- **LR Exp 3 (pop=10, mut=0.25):** atingiu 0.8045 na geração 11/20, com maior instabilidade (média caiu a 0.56 na geração 8 por mutação alta)
- **RF Exp 3 (pop=10, mut=0.25):** atingiu 0.7906 na geração 12/20 — exploração agressiva superou o Exp 1

Resultados — Comparativo de Métricas (Teste)

Modelo	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression (GA)	0.6989	**0.8125**	**0.7514**	0.8030
Random Forest (GA)	**0.7255**	0.7708	0.7475	**0.8136**
Logistic Regression (Original)	0.7397	0.7458	0.7427	0.8132
Random Forest (Original)	0.7114	0.7292	0.7202	0.7997

Análise de Melhorias (GA vs Original):

Métrica	LR: Original → GA	RF: Original → GA
Precision	0.7397 → 0.6989 (**-5.51%**)	0.7114 → 0.7255 (**+1.98%**)
Recall	0.7458 → 0.8125 (**+8.94%**)	0.7292 → 0.7708 (**+5.71%**)
F1-Score	0.7427 → 0.7514 (**+1.17%**)	0.7202 → 0.7475 (**+3.79%**)
ROC-AUC	0.8132 → 0.8030 (-1.26%)	0.7997 → 0.8136 (**+1.73%**)

Observações clínicas:

- O GA na **Logistic Regression** aumentou o Recall em **+8.94%** (de 0.7458 para 0.8125), detectando mais pacientes com AVC, ao custo de uma queda de -5.51% na Precision — trade-off aceitável no contexto clínico, onde falsos negativos são mais perigosos que falsos positivos
- O GA no **Random Forest** melhorou **todas as métricas** simultaneamente, com destaque para Recall +5.71% e F1 +3.79%
- A matriz de confusão do melhor modelo (LR GA) mostra 195 verdadeiros positivos e 45 falsos negativos em 240 casos de AVC

Ranking Completo — Todos os Modelos (por média de Recall + F1)

Rank	Modelo	Recall	F1-Score	Média
1º	LR — Exp1 (sklearn-genetic-opt)	0.8125	0.7514	0.7820
2º	LR — Exp2 (GA Manual, pop=30, mut=0.05)	0.8000	0.7456	0.7728
3º	LR — Exp3 (GA Manual, pop=10, mut=0.25)	0.8000	0.7456	0.7728
4º	RF — Exp3 (GA Manual, pop=10, mut=0.25)	0.7708	0.7475	0.7592
5º	RF — Exp1 (sklearn-genetic-opt)	0.7542	0.7418	0.7480
6º	RF — Exp2 (GA Manual, pop=30, mut=0.05)	0.7583	0.7368	0.7476
7º	LR — Original (sem GA)	0.7458	0.7427	0.7443
8º	RF — Original (sem GA)	0.7292	0.7202	0.7247

Destaques do ranking:

- Todos os 6 modelos otimizados por GA superaram os 2 modelos originais
- O melhor modelo (LR Exp1, `sklearn-genetic-opt`) teve Recall 8.94% superior ao LR Original
- As implementações manuais (Exp 2 e Exp 3) convergiram para resultados muito próximos da biblioteca, validando a implementação *from scratch*

Importância das Features (Permutação — Melhor Modelo GA)

As 5 features mais relevantes para o modelo de Logistic Regression (GA):

Feature	Importância	Desvio Padrão
`RIDAGEYR_age` (Idade)	0.0727	0.0131
`BPQ020_high_bp_bin` (Hipertensão)	0.0122	0.0049
`OCQ260_occupation` (Situação profissional)	0.0095	0.0038
`LBXGH_hba1c` (Hemoglobina glicada)	0.0053	0.0029
`BPXSY1_sbp` (Pressão sistólica)	0.0028	0.0040

A **idade** é a feature dominante, com importância ~6× superior às demais, consistente com a literatura médica sobre fatores de risco de AVC.

Integração com LLMs para Interpretação de Resultados

- Integrar uma LLM pré-treinada (GPT, Falcon, LLaMA, etc.) para:
  - Gerar explicações em linguagem natural dos diagnósticos produzidos pelos modelos;
  - Transformar dados numéricos e estatísticos em insights acionáveis para médicos;
  - Preparar a base para a futura integração com dados textuais no Módulo 3.
- Implementar técnicas de prompt engineering para obter respostas relevantes e adequadas ao contexto médico;
- Avaliar a qualidade das interpretações geradas.

Referências

1. CDC NHANES — https://www.cdc.gov/nchs/nhanes
2. Documentação scikit-learn — https://scikit-learn.org
3. Pandascouple. Projeto ML Previsão de AVC — https://pandascouple.medium.com/projeto-machine-learning-previs%C3%A3o-de-avc-f4b7dce11929
4. USP Rádio. Uso de IA e análise de dados na prevenção de AVC — https://jornal.usp.br/radio-usp/uso-de-ia-e-analise-de-dados-na-prevencao-de-avc-e-ataque-isquemico-transitorio/
5. Nature Research. Machine learning para risco cardiovascular — https://www.nature.com/articles/s41598-024-61665-4
6. Journal of Health Informatics. AVC diagnosis — https://jhi.sbis.org.br/index.php/jhi-sbis/article/view/980
7. Nature Research 2025. AVC modeling — https://www.nature.com/articles/s41598-025-01855-w
8. Atlas Grand Challenge - https://atlas.grand-challenge.org/
9. Notebook Atlas Grand Challenge - https://github.com/npnl/isles_2022/blob/main/ISLES_Example.ipynb
10. EfficientNetB0-Stroke Prediction-98.72 https://www.kaggle.com/code/abdallahwagih/efficientnetb0-stroke-prediction-98-72